JOUR 5

Exploiter, sauvegarder, passer au cloud

Vendredi 23 octobre 2026 - 9h10-17h10 - Infrastructure, orchestration et automatisation

Mastère Data Engineering & IA, M2 - EFREI Bordeaux - Alexandre STOPPINI

AU PROGRAMME

- Un objectif de service, et l'alerte qui réveille à bon escient
- Les données : les sauvegarder, tout perdre, puis les restaurer
- VéloCité sur un vrai cloud, avec une couche que vous écrivez, puis plus rien

JOUR 5

Plan de la journée

390 minutes utiles - quatre modules - quatre ateliers - trois démos en animation

Créneau	Bloc	Contenu
09h10 - 09h35	Dépôt, QCM	Le dépôt du jour et la correction, puis le QCM sur la journée 4
09h35 - 10h25	Module 11	Un objectif de service, l'alerte qui en découle, l'animation D18
10h40 - 11h18	Atelier 9	Les alertes SLO sur VéloCité (après la pause de 10h25)
11h18 - 11h33	Module 11, suite	Quand l'alerte sonne : l'incident, les rôles, le postmortem
11h33 - 11h48	Atelier 9 bis	Faire rédiger le postmortem par une IA, puis le faire corriger
11h48 - 12h10	Module 12	Les données survivent-elles ? L'animation D19
12h10 - 12h40	Atelier 10	La donnée du jour, la panne, la restauration
13h40 - 14h29	Module 13	Du poste au cloud, l'animation D20 (déjeuner de 12h40 à 13h40)
14h29 - 15h25	Atelier 11	La couche cloud, puis monter et déployer (pause 15h00 - 15h15)
15h25 - 15h55	Module 14	Pendant la construction des images : montée en charge, traces
15h55 - 16h55	Atelier 11, suite	Vérifier, prouver, détruire
16h55 - 17h10	Bilan	La semaine, le tag semaine, le 30 novembre

JOUR 5

Le dépôt du jour, puis le QCM

La correction des ateliers 1 à 8 d'abord, le QCM pendant que tout redémarre

Le QCM, après l'étape 4

- Lien sur la page Moodle du module
- Seul, sans document : 15 minutes
- Après l'envoi : "Afficher le score", lisez les justifications
- Ne compte pas dans la note
- L'étape 5, la vérification : en ouvrant l'atelier 9

À lire ; à copier depuis Commandes_J5, étapes 0 à 5

```
# 1. garder votre travail dans l'historique
git add -A
git commit -m "Avant la correction du J5"
# 2. les fichiers du jour
git pull --no-rebase --no-edit cours j5
# 3. la correction (Moodle) : Windows, puis macOS et Linux
Expand-Archive -Force "$HOME\Downloads\correction_J5.zip" .
unzip -o ~/Downloads/correction_J5.zip -d .
git add -A
git commit -m "Correction des ateliers 1 à 8"
git push
# 4. appliquer ; le QCM pendant que ça redémarre
kubectl apply -k deploy/overlays/dev
kubectl apply -k deploy/overlays/prod
kubectl -n velocite-dev apply -f workflows
tofu -chdir=infra/10-outils apply
# 5. en ouvrant l'atelier 9, tout OK (pwsh sur Mac)
./scripts/verifier.ps1 -Atelier 7,8
```

MODULE 11

Un objectif de service, et l'alerte qui en découle

- SLI, SLO, budget d'erreur : un chiffre pour arbitrer
- Le taux de consommation : à quelle vitesse on dépense ce budget
- Une alerte à deux fenêtres, écrite ligne par ligne, puis la vôtre à l'atelier 9
- Quand l'alerte sonne : l'incident, les rôles, le postmortem fait rédiger par une IA

MODULE 11

Les mots d'abord

Trois mots nouveaux, une ligne chacun

Mot	Ce que c'est, en une ligne
Budget d'erreur	La part d'échecs que l'objectif tolère : avec 99 %, 1 % des appels sur 30 jours
Taux de consommation	La vitesse à laquelle on dépense ce budget : 1, c'est le rythme prévu
Alerte à deux fenêtres	Une alerte qui regarde une période courte et une longue à la fois

MODULE 11

Le budget d'erreur, à calculer

Un objectif de 99 % sur 30 jours : combien de minutes d'échecs tolère-t-il ?

1. **Combien de minutes dans 30 jours ?**
2. 30 x 24 x 60 = 43 200 minutes
3. **99 % de réussite : combien de minutes d'échecs ?**
4. 1 % de 43 200 = 432 minutes, soit 7 h 12
5. **Une coupure totale d'une heure : quelle part du budget ?**
6. 60 / 432 = 14 % du budget du mois
7. À vous, au tableau : avec un objectif de 99,9 %, quel budget par mois ?
   Une coupure totale d'une heure tient-elle dedans ?

MODULE 11

Le taux de consommation, pas à pas

À quelle vitesse dépense-t-on le budget ? Un rapport entre deux parts d'échecs

Taux de consommation = part d'échecs observée / part d'échecs tolérée (1 %)

1. **1 % des appels échouent, en continu**
2. taux 1 : le budget dure exactement 30 jours
3. **2 % des appels échouent, en continu**
4. taux 2 : il est épuisé en 30 / 2 = 15 jours
5. **14,4 % des appels échouent**
6. taux 14,4 : épuisé en 2,1 jours ; en une heure, 14,4 / 720 = 2 % du budget
7. Le seuil des équipes d'exploitation : à partir de 14,4, on réveille quelqu'un, même la nuit.
   Source : Google, The Site Reliability Workbook, chap. 5, Alerting on SLOs
8. À vous : 5 % d'échecs pendant une heure. Quel taux ? Quelle part du budget du mois ?

MODULE 11

Ce qu'on fait du budget

Un chiffre arbitre entre l'équipe qui livre et l'équipe qui exploite

Budget confortable

- On livre, on met à jour
- On prend des risques mesurés
- On essaie la nouvelle version en prod

Budget presque épuisé

- On gèle les mises à jour non urgentes
- On travaille la fiabilité
- La décision se prend sur un chiffre, pas une opinion

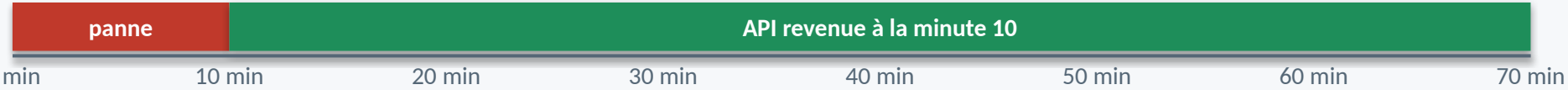
MODULE 11

Une alerte qui réveille à bon escient

Une coupure totale de 10 minutes, vue à travers trois alertes différentes

Une fenêtre, ce sont les X dernières minutes. On y calcule la part d'échecs, puis on la compare au seuil : 14,4 %.

1



2 **Fenêtre de 5 min**



Minute 1 : 1 min de panne sur 5 = 20 %, elle part. Un simple pic d'une minute la ferait donc partir. Minute 15 : la panne est sortie des 5 dernières minutes, elle retombe.

3 **Fenêtre de 1 h**



Minute 5 : 5 sur 60 = 8 %, rien. Minute 9 : 9 sur 60 = 15 %, elle part. Minute 30 : 10 sur 60 = 17 %, elle sonne encore, l'API est revenue depuis 20 min.

4 **Les deux, avec and**



Elle part quand les deux sont au-dessus du seuil (minute 9), et retombe dès que la fenêtre courte redescend (minute 14).

MODULE 11

La démo D18, scène par scène

Deux fenêtres sur une salve d'erreurs, dans Prometheus : capturé le 22 octobre

	La commande clé (complète dans l'animation)	Ce que la vraie sortie a montré
1 Le décor	`Mesurer`	Courte 0, longue 0, rien pour les deux réunies par and.
2 La salve	`1..250 | ForEach-Object { curl.exe -s -o NUL http://localhost:9090/api/v1/query }`	250 requêtes mal formées, une toutes les 100 ms : une erreur 400 chacune, 37 s.
3 La courbe sur 30 s	`sum(rate(prometheus_http_requests_total {code="400"}[30s]))`	Au-dessus du seuil (1 par seconde) dès 15,8 s ; 6,6 erreurs par seconde à 30,8 s.
4 La courbe sur 3 min	`sum(rate(prometheus_http_requests_total {code="400"}[3m]))`	Elle monte lentement et passe le seuil à 35,8 s (1,2).
5 Les deux, avec and	`<courte> > 1 and <longue> > 1`	Active de 35,8 à 60,8 s seulement : l'alerte partirait.
6 La fin de la salve	`Wait-Job $global:jobSalve`	Courte à 0 dès 65,8 s ; longue vers 1,46 jusqu'à 180,8 s, à 0 à 215,8 s.

MODULE 11

De l'alerte de jeudi à l'alerte SLO, écrite

Sur le compteur de la démonstration, prometheus_http_requests_total : la forme, pas la réponse

règles de la démonstration, en YAML

```yaml
- alert: TropDEchecs
  expr: sum by (handler) (rate(prometheus_http_requests_total{code=~"5.."}[5m]))
      / sum by (handler) (rate(prometheus_http_requests_total[5m])) > 0.01
  for: 2m
- record: demo:echecs:ratio_rate5m
  expr: sum by (handler) (rate(prometheus_http_requests_total{code=~"5.."}[5m]))
      / sum by (handler) (rate(prometheus_http_requests_total[5m]))
- record: demo:echecs:ratio_rate1h
  expr: sum by (handler) (rate(prometheus_http_requests_total{code=~"5.."}[1h]))
      / sum by (handler) (rate(prometheus_http_requests_total[1h]))
- alert: BudgetConsommeVite
  expr: demo:echecs:ratio_rate1h > 14.4 * 0.01
    and demo:echecs:ratio_rate5m > 14.4 * 0.01
```

1. Jeudi : plus de 1 % d'échecs sur 5 min, tenu 2 min. Un pic d'une minute reste 5 min dans la fenêtre : elle part.
2. record range la part d'échecs sous un nom ; Prometheus la recalcule à chaque évaluation.
3. Le même calcul sur [1h] : un second nom, une seconde série.
4. and : les deux noms au-dessus de 14,4 %. Pas de for : l'heure fait déjà attendre.

5. Le test rejoue en une seconde une heure de trafic normal, puis 10 minutes de panne. À quels trois instants vérifier, et qu'attendre à chacun ? C'est l'atelier 9.

ATELIER 9

Atelier 9 - Écrire, tester, charger, provoquer

38 min, de 10h40 à 11h18 - en binôme - les commandes : Fiche_Atelier9_J5, mêmes numéros

Consignes, dans l'ordre

1. Dans observability/regles/enregistrement.yaml, écrivez deux règles d'enregistrement : la part d'échecs du client témoin, par namespace et par route, sur 5 minutes (velocite:echecs_client:ratio_rate5m), puis sur 1 heure (velocite:echecs_client:ratio_rate1h).
2. Dans observability/regles/alertes.yaml, remplacez DisponibiliteSousObjectif par BudgetErreurConsommeVite : elle part quand les deux parts dépassent 14,4 fois 1 %, sans for.
3. Dans observability/tests/slo_test.yaml, complétez les trois vérifications : à 65 minutes, aucune alerte ; à 72 minutes, l'alerte ; à 76 minutes, plus rien.
4. Vérifiez la syntaxe des deux fichiers de règles, puis lancez le test : il doit afficher SUCCESS.
5. Chargez les règles dans Prometheus avec tofu apply sur infra/10-outils, puis vérifiez que BudgetErreurConsommeVite apparaît dans la page Alerts.
6. Coupez l'API de dev pendant 10 minutes. Notez l'heure à laquelle l'alerte part. Remettez l'API, puis notez l'heure à laquelle l'alerte retombe.
7. Pendant l'attente, écrivez dans docs/RUNBOOK.md la section Budget d'erreur consommé vite : ce qu'on voit, ce qu'on vérifie, ce qu'on fait.
8. Validez avec ./scripts/verifier.ps1 -Atelier 9 : tout doit être OK.

Les commandes de chaque étape, pour Windows, macOS et Linux, sont dans Fiche_Atelier9_J5, avec les mêmes numéros, ainsi que la table Si ça coince. Pour valider l'atelier : ./scripts/verifier.ps1 -Atelier 9 (pwsh devant sous macOS et Linux).

MODULE 11

Quand l'alerte sonne : un incident, étape par étape

Une nuit en prod : la version 2.3.0, livrée à 2 h 04, fait échouer tous les appels

1 Détecter	2 Qualifier	3 Atténuer	4 Résoudre	5 Postmortem
2 h 14 - 2 h 16	**2 h 16 - 2 h 21**	**2 h 24 - 2 h 29**	**le lendemain**	**dans la semaine**
L'alerte part. Quelqu'un accuse réception en 2 minutes ; sinon, elle passe à la personne suivante.	Toute la prod est touchée, depuis 2 h 05. Chaque minute de panne coûte 1 / 432 du budget du mois.	Retour arrière vers la 2.2.1 : le service revient à 2 h 29, l'alerte retombe à 2 h 33. On comprend ensuite.	La cause est corrigée dans le dépôt, puis livrée par la chaîne habituelle. Rien à la main dans le cluster.	Le récit écrit de l'incident, sans coupable, avec ses actions : la slide 15.

6 Les chiffres de cet incident : détection en 9 min (2 h 05 à 2 h 14) ; rétablissement en 24 min (2 h 05 à 2 h 29), dont la moyenne sur les incidents s'appelle le MTTR ; budget consommé : 24 / 432, soit 5,6 % du mois.

MODULE 11

Qui fait quoi pendant un incident

Trois rôles, puis ce qu'on dit aux usagers, en trois phrases

Trois rôles

- **Le responsable de l'incident** : tient l'état, décide, répartit le travail ; il ne tape pas les commandes
- **La personne qui agit** : la seule à modifier le système, à la demande du responsable
- **La personne qui communique** : informe l'équipe et les usagers, à heure fixe
- Seul de garde, vous tenez les trois : décidez d'abord, puis agissez

Le message aux usagers de VéloCité

"Depuis 2 h 05, l'application VéloCité ne permet plus de consulter les stations ni vos trajets. Nos équipes sont sur l'incident. Prochaine information à 2 h 45 au plus tard."

- Ce qui ne marche pas, depuis quand, la prochaine nouvelle
- Ni jargon, ni cause supposée ; une heure promise, puis tenue

MODULE 11

Le postmortem, sans coupable

Le récit écrit de l'incident, pour qu'il ne se reproduise pas

Les sections, dans l'ordre

- Résumé, en deux phrases
- Impact chiffré : durée, usagers, minutes de budget
- Chronologie, heure par heure
- Causes : "pourquoi ?" répété, sans nom de personne
- Ce qui a bien marché
- Actions : un responsable, une date

Exemple : la 2.2.0 de mercredi, sans TRIPS_CACHE_TTL

- **Résumé** : la 2.2.0 exige TRIPS_CACHE_TTL ; sans elle, son pod sort en code 78
- **Impact** : aucun appel perdu, la 2.1.0 a servi tout du long ; 0 minute de budget
- **Causes** : la variable est nouvelle ; rien ne vérifie l'overlay avant le déploiement
- **Bien marché** : la sonde de disponibilité a gardé l'ancienne version en service
- **Action** : faire échouer le déploiement si une variable exigée manque dans l'overlay ; l'équipe exploitation, d'ici le 29 octobre

ATELIER 9 BIS

Atelier 9 bis - Faire rédiger le postmortem par une IA

15 min, de 11h33 à 11h48 - en binôme - fiche : Fiche_Atelier9bis_J5, mêmes numéros

Consignes, dans l'ordre

1. Ouvrez Dossier_Incident_2.3.0.md (Moodle) : repérez ses dix pièces, sans chercher encore la cause.
2. Dans l'IA de votre choix, en version gratuite, écrivez un prompt qui lui fait rédiger le postmortem, puis collez le dossier à la suite.
3. Rangez sa réponse dans docs/POSTMORTEM.md, et votre prompt en "Annexe A - Le prompt utilisé".
4. Vérifiez au moins cinq de ses affirmations contre le dossier, corrigez ce qui est faux, notez chaque vérification en "Annexe B".
5. Dans une nouvelle conversation, collez Prompt_Correction_Postmortem.md, puis tout votre fichier.
6. Reprenez chaque KO avec votre IA de rédaction, puis renvoyez le fichier, jusqu'à "Verdict global : OK".
7. Commitez et poussez docs/POSTMORTEM.md.

Ici, l'IA est autorisée : la compétence, c'est de faire rédiger puis de vérifier. La correctrice ne donne jamais la réponse. Cet atelier ne compte pas dans la note.

Les données survivent-elles ?

- Ce que up.ps1 reconstruit et ce qu'il ne reconstruit pas
- Sauvegarder, tout perdre, restaurer : la séquence
- RPO et RTO : combien de données on perd, combien de temps on attend

MODULE 12

Les mots d'abord

Cinq mots pour protéger les données, une ligne chacun

Mot	Ce que c'est, en une ligne
Sauvegarde logique	Une copie des données sous forme de commandes SQL, rejouables sur une base
pg_dump	L'outil de PostgreSQL qui écrit cette sauvegarde, sans arrêter la base
Volume persistant (PVC)	Le disque que Kubernetes garde quand le pod redémarre, pas quand on le supprime
RPO	Recovery Point Objective : combien de données on accepte de perdre
RTO	Recovery Time Objective : combien de temps on accepte d'attendre le retour

MODULE 12

Ce que up.ps1 ne reconstruit pas

Si le cluster disparaît : ce qui revient depuis le dépôt et ce qui est perdu

1 **Le cluster, les outils, l'environnement**	décrits dans le dépôt : up.ps1 les recrée à l'identique
2 **Les secrets**	régénérés par OpenTofu : neufs, mais tout fonctionne
3 **Les données de démonstration**	rechargées par l'image de la base, au premier démarrage
4 **Les données écrites depuis**	le bilan du 12 mars, les comptes, les trajets : perdus

5 Les données écrites depuis vivent dans un seul endroit : le volume donnees-postgres-0. Le supprimer, c'est les perdre. Une sauvegarde doit donc vivre hors du cluster.

MODULE 12

Sauvegarder, puis restaurer : la séquence

Six gestes, dans l'ordre ; vous les écrivez dans deux scripts à l'atelier 10

1 — pg_dump dans le pod

Avec l'option --clean, la base écrit dans /tmp du pod une sauvegarde qui sait effacer avant de recréer.

2 — kubectl cp vers le poste

Le fichier sort du cluster et arrive dans sauvegardes/, que Git ignore.

3 — L'incident

Le StatefulSet et le volume donnees-postgres-0 sont supprimés ; la base renaît sans vos données.

4 — kubectl cp vers le pod

Le fichier revient dans /tmp du pod de la nouvelle base.

5 — psql -f

La base rejoue la sauvegarde : les tables sont recréées, les lignes remises.

6 — Vérifier

La donnée du jour : 250 trajets avant, 0 après la panne, 250 et la même empreinte après.

7 Le fichier ne passe jamais par un pipe PowerShell : sous Windows PowerShell 5.1, les accents seraient abîmés. Toujours kubectl cp, dans un sens comme dans l'autre.

MODULE 12

La démo D19, scène par scène

Sauvegarder la base, la perdre, la restaurer : capturé le 22 octobre

	La commande clé (complète dans l'animation)	Ce que la vraie sortie a montré
1 La base	`kubectl exec postgres-0 -- psql -c "SELECT count(*) FROM emprunts"`	1000 emprunts dans la base de démonstration.
2 L'activité du jour	`kubectl exec postgres-0 -- psql -c $sql`	INSERT 0 250 : 1250 emprunts.
3 La sauvegarde	`pg_dump --clean --if-exists -f /tmp/biblio.sql` `kubectl cp postgres-0:/tmp/biblio.sql sauvegardes/biblio.sql`	biblio.sql sur le poste : 57 754 octets, les 250 emprunts du jour dedans. 1,5 s.
4 L'incident	`kubectl delete statefulset postgres` `kubectl delete pvc donnees-postgres-0`	Base et volume supprimés, puis recréés par apply -k : 1000. Les 250 du jour sont perdus. 12,7 s.
5 La restauration	`kubectl cp sauvegardes/biblio.sql postgres-0:/tmp/biblio.sql` `psql -v ON_ERROR_STOP=1 -f /tmp/biblio.sql`	COPY 1250 : la sauvegarde est rejouée. 0,7 s.
6 La preuve	`kubectl exec postgres-0 -- psql -c $sql`	1250 emprunts ; Thérèse Raquin 250, accents intacts.

MODULE 12

RPO et RTO : combien perd-on, combien de temps

Deux chiffres qui disent si une stratégie de sauvegarde suffit

RPO : ce qu'on accepte de perdre

- Une sauvegarde chaque nuit, à 2 h
- Une panne à 18 h : tout ce qui a été écrit depuis 2 h est perdu, soit 16 heures
- Au pire, juste avant la sauvegarde suivante : 24 heures
- À vous : une sauvegarde toutes les 6 heures, quel RPO ?

RTO : le temps pour revenir

- Détecter : l'alerte, ou un usager qui se plaint
- Décider : restaurer, et depuis quelle sauvegarde
- Restaurer, puis vérifier en comptant
- Mesuré à l'atelier 10 : la restauration seule ; le vrai RTO est plus long

ATELIER 10

Atelier 10 - Sauvegarder, subir la panne, restaurer

30 min, de 12h10 à 12h40, Si ça coince compris - en binôme - Fiche_Atelier10_J5

Consignes, dans l'ordre

1. Vérifiez que la base de dev tourne : postgres-0 doit être prêt.
2. Lancez donnees-du-jour.ps1, puis notez dans docs/RUNBOOK.md le nombre de trajets et l'empreinte qu'il affiche.
3. Complétez sauvegarder.ps1, lancez-le, puis vérifiez que git status ne montre pas sauvegardes/.
4. Lancez panne.ps1 et tapez oui : la base et son disque sont détruits.
5. Recréez une base vide avec kubectl apply -k, puis constatez avec donnees-du-jour.ps1 -Verifier que la donnée du jour a disparu.
6. Complétez restaurer.ps1, puis restaurez votre sauvegarde en chronométrant : cette durée est votre RTO.
7. Relancez donnees-du-jour.ps1 -Verifier, comparez avec ce que vous avez noté, puis terminez la section Restauration des données du runbook.
8. Validez avec ./scripts/verifier.ps1 -Atelier 10 : les neuf contrôles doivent être OK.

Les commandes de chaque étape, pour Windows, macOS et Linux, sont dans Fiche_Atelier10_J5, avec les mêmes numéros, ainsi que la table Si ça coince. Après la panne, donnees-du-jour.ps1 se lance toujours avec -Verifier.

MODULE 13

Du poste au cloud

- Le même dépôt, une autre couche 00 : k3s sur une machine louée à l'heure
- Qui est responsable de quoi ; ce que votre token vous laisse faire
- Le réseau pièce par pièce, ce qui casse sur un cloud, ce que ça coûte

MODULE 13

Les mots d'abord

Huit mots pour le cloud, une ligne chacun

Mot	Ce que c'est, en une ligne
Cloud public	Des machines louées à un fournisseur, payées à l'heure, pilotées par API
Projet	Chez Hetzner, un espace isolé : ses machines, ses pare-feux, sa facture
Token d'API	Le secret qui ouvre le projet de l'équipe à OpenTofu : HCLOUD_TOKEN
Pare-feu	La liste des ports ouverts en entrée, et depuis quelles adresses
k3s	Un Kubernetes en un seul programme, pour une seule machine
NodePort	Un Service publié sur un port de la machine (30000 à 32767)
Responsabilité partagée	Ce que le fournisseur garantit et ce qui reste à vous
Service géré	Le fournisseur fait tourner le logiciel à votre place

MODULE 13

Le même dépôt, une autre couche 00

infra/00-cloud-hetzner remplace infra/00-cluster ; vous l'écrivez, le reste est identique

	Sur le poste	Chez Hetzner
Le cluster	kind, deux conteneurs	k3s, installé par votre cloud-init
Créé par	infra/00-cluster, fournie	infra/00-cloud-hetzner, écrite par vous
Les images	construites sur le poste	construites sur la machine
L'entrée	port 8443 du poste	port 30443 de la machine, à votre adresse
State de la couche 00	fichier local chiffré	fichier local chiffré
infra/20-env, deploy/	inchangés	inchangés

MODULE 13

Qui est responsable de quoi

Le partage dépend du service choisi ; les trois dernières lignes ne changent jamais

Couche	Serveur + k3s (cet après-midi)	Kubernetes géré
Bâtiment, matériel, hyperviseur	Hetzner	le fournisseur
Système de la machine, mises à jour	vous	le fournisseur (plan de contrôle) ; vous (nodes)
Kubernetes : version, certificats	vous	le fournisseur
Pare-feu, ports ouverts	vous	vous
Tokens et droits d'accès	vous	vous
Applications, données, secrets	vous	vous

MODULE 13

Le projet de l'équipe, son token

Un projet Hetzner par équipe, un token par projet : ce qu'il ouvre, et où il vit

Ce que le token vous laisse faire

- Créer et détruire des machines, des pare-feux, des clés, dans le projet de l'équipe
- Rien dans le projet d'une autre équipe : vous ne le voyez même pas
- Tout ce que vous créez est facturé à l'intervenant, à l'heure
- Il est supprimé ce soir à 18h, avec ce qui reste

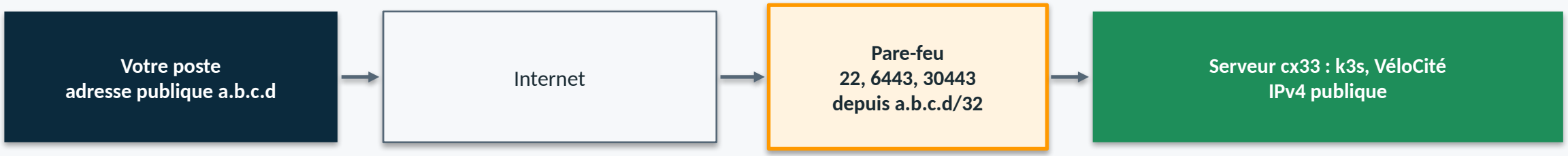
Où il vit, où il ne vit jamais

- Dans $env:HCLOUD_TOKEN, le temps du terminal
- Jamais dans un fichier du dépôt, ni dans un commit
- Jamais dans une capture d'écran ni un message public
- Un token poussé est un token perdu : prévenez-moi, je le révoque

MODULE 13

Le réseau, pièce par pièce

Ce que vous écrivez cet après-midi dans infra/00-cloud-hetzner/reseau.tf



1. La machine a une adresse publique : sans pare-feu, tout Internet joint chacun de ses ports
2. hcloud_firewall : trois règles en entrée, chacune depuis var.mon_ip seulement ; la sortie reste ouverte
3. 22 : SSH : cloud.ps1 envoie le code, construit les images
   6443 : l'API Kubernetes : kubectl et OpenTofu depuis votre poste
   30443 : le frontal de VéloCité, en NodePort, comme sur kind
4. À l'école, toute la promotion sort par la même adresse : votre pare-feu laisse entrer vos camarades de promotion, pas le reste d'Internet. Chez vous, l'adresse change : relancez monter, la règle se met à jour.

MODULE 13

Ce qui casse sur un cloud et que le poste cachait

Cinq erreurs et ce qui les empêche dans votre dépôt

Erreur	Ce qui se passe	Ce qui l'empêche ici
6443 ouvert à 0.0.0.0/0	l'API sondée en quelques minutes	var.mon_ip validée en /32
Token dans un dépôt	des machines de minage sur la carte	variable d'environnement ; verifier.ps1
Machine éteinte, pas détruite	facturée tant qu'elle existe	cloud.ps1 -Etape detruire
Clé SSH dans le dépôt	la machine à qui la lit	.cles/ ignoré par Git
Adresse changée (Wi-Fi)	plus rien ne répond, sans erreur	relancer monter

MODULE 13

Ce que ça coûte

Prix Hetzner relevés le 22 octobre 2026, hors taxes

1 **Serveur cx33 (4 vCPU, 8 Go)**	0,0143 euro de l'heure, plafonné à 8,49 euros par mois
2 **Adresse IPv4 publique**	environ 0,0007 euro de l'heure, facturée à part
3 **Une heure commencée**	due en entier
4 **Machine éteinte**	facturée comme allumée, tant qu'elle existe
5 **L'atelier d'une équipe, 3 h**	environ 5 centimes ; 8 machines oubliées 24 h : 3 euros

6 La bonne question n'est pas "lequel est le moins cher", mais "qui, dans l'équipe, fera les mises à jour à 3 h du matin". Une machine nue coûte peu et demande tout ; un service géré coûte plus et en demande moins.

MODULE 13

La démo D20, scène par scène

Une machine louée, vue par l'API comme vous la verrez : capturé le 22 octobre

	La commande clé (complète dans l'animation)	Ce que la vraie sortie a montré
1 La machine	`(Invoke-RestMethod "$api/servers" -Headers $h).servers`	Ma machine de test : running, cx33, une adresse publique.
2 Le processeur	`Invoke-RestMethod "$api/servers/$($srv.id)/metrics?$q" -Headers $h`	Jusqu'à 199,4 % d'un vCPU pendant la construction des images.
3 Le pare-feu	`(Invoke-RestMethod "$api/firewalls" -Headers $h).firewalls.rules`	Trois règles en entrée, 22, 6443 et 30443, chacune depuis une seule adresse.
4 Le node	`kubectl --context $ctx get nodes`	Un node, Ready : k3s v1.36.5 tourne sur la machine.
5 Les pods	`kubectl --context $ctx -n velocite-dev get pods`	Le même overlay que sur le poste : 7 pods Running, l'initialisation Completed.
6 Vérifier	`.\scripts\cloud.ps1 -Etape verifier`	{"environnement":"dev","service":"velocite-api","version":"2.2.0"}

ATELIER 11

Atelier 11, partie 1 - La couche cloud, en code

31 min, de 14h29 à 15h00 - en équipe, sur un seul poste - hors grille - Fiche_Atelier11_J5

Consignes, dans l'ordre

1. Ouvrez infra/00-cloud-hetzner : les blocs sont écrits. Repérez les valeurs marquées A FAIRE, numérotées de 1.a à 3.c, et les garde-fous en bas des fichiers.
2. Pièce 1, reseau.tf : les trois ports (1.a) et l'adresse autorisée (1.b).
3. Pièce 2, calcul.tf : de 2.a à 2.g, des variables et des id, écrits sans guillemets.
4. Pièce 3, user_data.sh.tftpl : la version de k3s, l'adresse du certificat, le fichier témoin (3.a à 3.c). Puis tofu fmt.
5. Dans le terminal du poste de l'équipe, saisissez le token de l'équipe (la saisie est masquée), l'identifiant velocite-equipeN et la phrase du state, puis testez le token.
6. Vérifiez avant de monter avec verifier.ps1 -Atelier 11 -AvantMonter, jusqu'à 12 contrôles, 0 KO. Rien n'est créé avant la pause.

Les commandes de chaque étape, pour Windows, macOS et Linux, sont dans Fiche_Atelier11_J5, avec les mêmes numéros. Chaque KO dit où regarder, par le numéro du commentaire, sans donner la valeur.

ATELIER 11

Atelier 11, partie 2 - Monter, puis lancer le déploiement

10 min, de 15h15 à 15h25 - en équipe, sur le poste de l'équipe - hors grille - Fiche_Atelier11_J5, partie 2

Consignes, dans l'ordre

1. Montez la machine avec cloud.ps1 -Etape monter : il relance la vérification et s'arrête au moindre KO, sans rien créer. Lisez le plan, puis tapez yes. k3s doit être prêt en 3 à 5 minutes ; sinon, regardez votre cloud-init.
2. Déployez avec -Etape deployer : les images sont construites sur la machine, comptez 10 à 15 minutes. Laissez ce terminal tourner, sans le fermer.

Pendant la construction des images : 30 minutes de cours, le module 14. À 15h55, retour à votre terminal pour la consigne 3. Les commandes sont dans Fiche_Atelier11_J5, partie 2, avec les mêmes numéros.

Monter en charge : le mécanisme

Le HorizontalPodAutoscaler de l'API de VéloCité en prod : 2 à 4 pods, cible 70 %

1 metrics-server

relève le CPU de chaque pod toutes les 15 s ; installé par infra/10-outils

2 HPA api

toutes les 15 s, compare la moyenne à la cible : 70 % de la request, soit 70m

3 Le calcul

désiré = ceil(actuel x utilisation / cible), borné entre 2 et 4

4 Deployment api

le HPA change replicas ; les pods neufs démarrent

5 Exemple : 2 pods à 140m de CPU pour une request de 100m, soit 140 %. ceil(2 x 140 / 70) = 4 pods, le maximum. Avec 4 pods, la même charge donne 70m chacun : 70 %, le calcul se stabilise.

6 Redescendre lentement : à la baisse, le HPA retient la plus haute valeur des 5 dernières minutes (300 s, par défaut et dans la prod). À la hausse, il n'attend pas.

7 Un pod neuf ne reçoit du trafic qu'une fois prêt : c'est la sonde de disponibilité de l'atelier 6 qui décide. Monter prend le temps du démarrage, pas 15 s.

MODULE 14

Monter en charge : ce qui se règle, ce qui casse

Quatre réglages, cinq pannes, toutes visibles sur VéloCité

Ce qui se règle

- minReplicas : le plancher, 2 en prod
- maxReplicas : le plafond, 4 en prod ; il protège le quota et la facture
- La cible : 70 % de la request de CPU
- La request de CPU : 100m pour l'API ; c'est elle qui donne le pourcentage

Ce qui casse

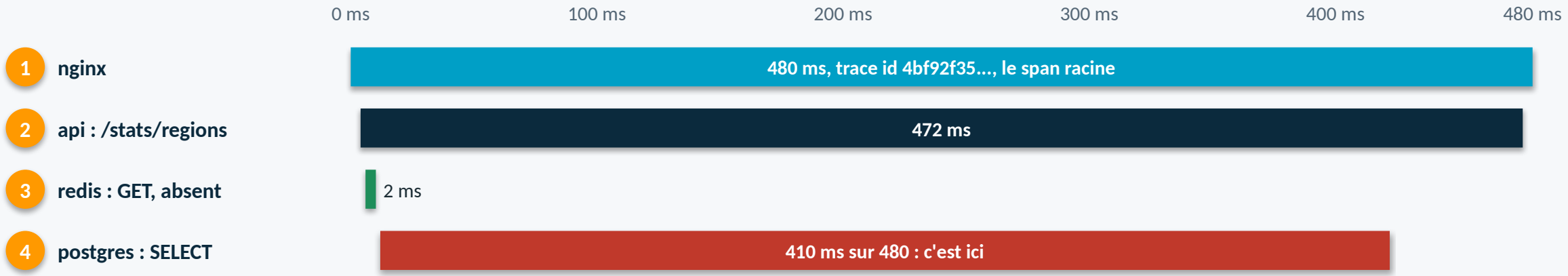
- Pas de request : le HPA affiche <unknown> et ne fait rien
- La base ne suit pas : Postgres reste à un pod, le goulot se déplace
- Le quota du namespace refuse les pods en trop
- Sur un cloud, plus de pods, c'est plus de machines, donc une facture
- Un pod lent à démarrer arrive après le pic

À vous : maxReplicas passe à 6, l'API monte à 6 pods, la base sature. Qu'a-t-on gagné ?

MODULE 14

Une requête, de bout en bout : les traces

GET /api/stats/regions dure 480 ms : où passe le temps ?



5 Le trace id voyage d'un service à l'autre dans un en-tête HTTP (norme W3C Trace Context) :

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```

version, trace id (32 caractères), span parent (16), options (01 : la trace est enregistrée)

6 Chaque service envoie ses spans à un collecteur OpenTelemetry ; Jaeger ou Grafana Tempo les rangent par trace id et les affichent en cascade, comme ici. VéloCité n'a pas de traces : on sait ce que c'est, on ne l'installe pas.

MODULE 14

Logs, métriques, traces : qui répond à quelle question

Trois outils, trois questions ; en incident, on les prend dans cet ordre

La question	Qui y répond	Ce que ça coûte	Chez VéloCité
Combien d'échecs, depuis quand ?	Les métriques : des nombres dans le temps	Peu : une série par étiquette	Prometheus
Pourquoi cette requête a-t-elle échoué ?	Les logs : une ligne par événement	Beaucoup de volume	Loki
Où passe le temps de cette requête lente ?	Les traces : une requête suivie	Du code dans chaque service	Non

En incident : la métrique alerte, la trace situe, le log explique. Le 30 novembre, dites lequel des trois vous regardez, et pourquoi.

ATELIER 11

Atelier 11, partie 2, suite - Vérifier, prouver, détruire

45 min, de 15h55 à 16h40, puis 15 de Si ça coince - en équipe - hors grille - Fiche_Atelier11_J5

Consignes, dans l'ordre

3. À 15h55, de retour à votre terminal : l'API Hetzner, le serveur et le pare-feu de l'équipe. Vérifiez que deployer s'est terminé sans erreur, puis demandez à l'API ce que vous avez créé.
4. Vérifiez avec -Etape verifier, puis prouvez que les garde-fous ont suivi : un pod root refusé, les NetworkPolicy présentes.
5. Détruisez avec -Etape detruire, depuis le poste qui a monté : l'API ne doit plus lister aucun serveur.
6. Vérifiez avec git status qu'il n'y a ni token ni clé, puis écrivez la section Sur Hetzner de DEPLOIEMENT.md.
7. Validez avec ./scripts/verifier.ps1 -Atelier 11 : tout doit être OK.

Les commandes de chaque étape, pour Windows, macOS et Linux, sont dans Fiche_Atelier11_J5, partie 2, avec les mêmes numéros. Vous n'avez pas accès à la console Hetzner : vous interrogez l'API avec le token de l'équipe.

ATELIER 11

Si ça coince

Ce qui peut arriver chez Hetzner, et la sortie

Ce qu'on voit	La cause	La sortie
Un KO avant de monter	une valeur manquante ou fausse	la ligne qui suit le KO dit où regarder
Test du token : 401	token mal collé, ou d'un autre projet	le saisir à nouveau ; me demander
resource_limit_exceeded	les 36 vCPU du compte sont pris	me prévenir
server type not available	emplacement plein	emplacement fsn1 (Fiche_Atelier11_J5)
k3s pas prêt après 8 minutes	une erreur dans votre cloud-init	ssh ; cloud-init-output.log
UNPROTECTED PRIVATE KEY FILE	droits de la clé sous Windows	relancer monter : il les corrige
verifier : connexion expirée	votre adresse a changé (Wi-Fi)	relancer monter
Serveur encore là après detruire	detruire interrompu	le relancer ; sinon, je le fais à 18h

BILAN

Ce que vous avez construit cette semaine

Cinq jours, cinq étages, et pour chacun une preuve qui se rejoue

LUNDI	MARDI	MERCREDI	JEUDI	VENDREDI
Des images sûres - L'inventaire - Multi-stage, non root - La CI : build, Trivy	**L'infra en code** - OpenTofu, state chiffré - Le cluster, ses outils - dev fermé	**Sur Kubernetes** - Base, overlays - Une règle réseau par service - Une version qui casse	**Décider, prévenir** - Le bilan qui décide - Alertes testées - Une vraie panne	**Mesurer, protéger** - Un SLO et son alerte - Sauvegarder, restaurer - Ailleurs, puis plus rien
Preuve : la CI verte	Preuve : plan : No changes	Preuve : diff vide	Preuve : 33, trois fois	Preuve : 250 trajets rendus, même empreinte

BILAN

La preuve de la semaine, et le tag

Les vérifications du jour, puis le tag semaine : l'état de fin de semaine

À lire ; à copier depuis Commandes_J5, Bilan

```
./scripts/verifier.ps1 -Atelier 9,10
./scripts/verifier.ps1 -Atelier 11
# tout OK
kubectl config get-contexts
# plus de contexte hetzner-...
git status
# ni token, ni clé, ni sauvegarde
git add -A
git commit -m "Ateliers 9 à 11"
git push
git tag semaine
git push origin semaine
```

En une phrase, pour la directrice financière

"Nous sommes prévenus quand le service se dégrade, avant les clients. Si les données disparaissent, nous les rendons en quelques minutes. Et la plateforme peut partir chez un autre hébergeur en une après-midi."

Le tag semaine fige la fin de semaine. up.ps1 et docs/ sont notés sur le tag demo du 30 novembre : d'ici là, complétez-les et poussez.

BILAN

Le 30 novembre : préparer son passage

Le matin, on finit et on monte ; l'après-midi, chacun démontre seul au clavier

La journée

- Le matin : finir up.ps1 et son contrôle, écrire le tableau des preuves, relire les documents
- Puis le montage depuis zéro sur votre poste, et le tag demo
- L'après-midi : les passages, 15 minutes chacun, sans pause
- Votre passage : démontage, remontage, une panne, l'alerte, le runbook

Ce qui est noté au passage

- Réplicabilité 6 : la plateforme complète renaît, le contrôle passe
- Supervision 5 : la panne, l'alerte, son temps
- Vulgarisation 4 : ce que votre geste protège, sans jargon
- Compréhension 5 : chaque ligne du dépôt, même celles d'un autre